Binary Hopfield Networks Training & Inference

May 19 Meeting - ECE SRIP - Code Review, Deep Dive & MNIST Demo

s - {-1, +1}

Today's Agenda

01

Code & Repo Overview

Python pipeline, C sim, RTL - what each file does

02

Datasets & Specs

What data we're using, format, size, evaluation protocol

03

Training Deep Dive

Hebbian rule, Storkey rule, comparison table

04

Inference & Convergence

Async/sync update, energy function, fixed-point detection

05

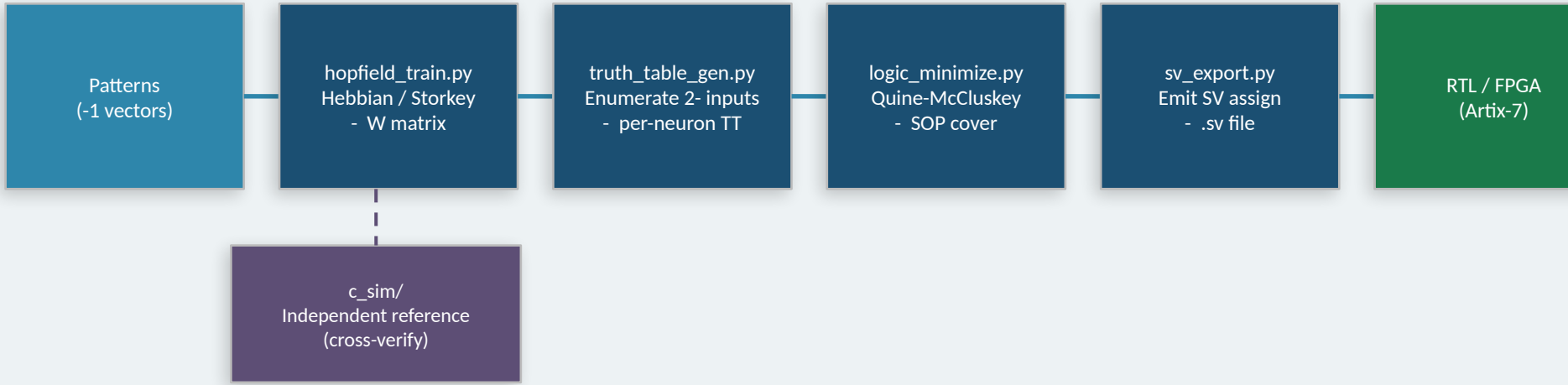
MNIST Visualization

Live demo: store digits - corrupt - retrieve - energy

01

Code & Repo Overview

01 - Code & Repo Overview - Pipeline at a Glance



- Python | C Sim | RTL Output

`python/hopfield_train.py - python/truth_table_gen.py - python/logic_minimize.py`

`python/sv_export.py - python/pipeline.py | c_sim/hopfield_train.c - hopfield_sim.c - hopfield.h`

`rtl/hopfield_top.sv - neuron_bank.sv - weight_rom.sv - update_ctrl.sv`

01 - Code Chunks - Specifications

File	Purpose	Key Functions	In - Out
hopfield_train.py	Hebbian & Storkey weight learning	train_hebbian() train_storkey() update_async() energy()	Pattern list - W.npy
truth_table_gen.py	Enumerate neuron update functions	enumerate_truth_tables() save_csv() save_json()	W.npy - TT CSV + JSON
logic_minimize.py	Q-M Boolean minimization + hazard-free cover	minimize_all() hazard_free_cover()	Truth tables - SOP cover
sv_export.py	SystemVerilog code generation	generate_sv()	Cover + TT - .sv file
pipeline.py	End-to-end driver script	stage_train/tt/minimize/sv()	CLI args - all artifacts
hopfield_train.c	C weight computation	hopfield_train_hebbian() hopfield_train_storkey()	Pattern arrays - W.txt
hopfield_sim.c	C simulation + demo main()	hopfield_run_async/sync() hopfield_energy()	W + state - converged state

02 - Datasets & Evaluation Protocol

Random Bipolar (Small)

```
N = 8 neurons - P = 3 patterns
Dtype: int8  -1  |  Seed: 42
```

Default pipeline demo, SV synthesis

Capacity: ~1 pattern (-=0.138)

Random Bipolar (Medium)

```
N = 16 neurons - P = 4 patterns
Dtype: int8  -1  |  Seed: 42
```

Scalability testing, C sim

Capacity: ~2 patterns

sklearn Digits (8-8)

```
N = 64 neurons - P = 5 patterns
Binarised MNIST-like  -1
```

MNIST demo, visual verification

Capacity: ~9 patterns

Custom Pattern File

```
.txt  -  one row per pattern
Space-separated -1 integers
```

Application-specific data

Variable by N and P

Evaluation: store P patterns - corrupt each with k bit-flips - run updates - check overlap m = (1/N) xi.s -> 1.0 = perfect recall

03

Training Deep Dive

03 - Hebbian Learning (Outer-Product Rule)

$$W = (1/N) * \sum_{\mu} \xi^{\mu} * (\xi^{\mu})^T \quad \text{with } W_{ii} = 0$$

Each term $\xi_i * \xi_j$ reinforces connections between neurons that agree in pattern μ. Diagonal zeroed - no self-excitation.

One-Shot

Full W computed in one pass over all patterns. O(N·P) time.

Capacity

$\alpha_{max} \approx 0.138$ → ~0.138N patterns reliably stored

Spurious States

Inverted patterns (-) and mixtures are also attractors

Hardware-Friendly

Simple outer-product sum; weights are integers if N is small

```
W = np.zeros((N, N))
for xi in patterns:  W += np.outer(xi, xi)
W /= N;  np.fill_diagonal(W, 0)
```

03 - Storkey Learning Rule (Higher Capacity)

```
W  <-  W  +  (1/N) [ xi*xi^T  -  h*xi^T  -  xi*h^T ]

where  h_i  =  Sum_{k!=i} W_ik * xi_k    (local field from current weights, excluding diagonal)
```

Correction terms h- + -h- subtract interference from previously stored patterns. Applied once per pattern, in order.

Hebbian	Storkey
One-shot (batch)	Sequential (online)
- - 0.138N	- - 0.14-0.22N
More spurious states	Fewer spurious states
Order-independent	Order-dependent
Simpler to implement	2- compute per pattern

03 - Training Methods Comparison

Property	Hebbian	Storkey	Pseudo-Inverse
Capacity (- = P/N)	~0.138	~0.14-0.22	1.0 (exact up to N)
Computation	O(N-P)	O(N-P)	O(N-P + inverse)
One-shot learning?	**YES**	**NO - Sequential**	**YES (batch)**
Error correction	None	Partial	Perfect
Spurious attractors	Many	Fewer	Fewest
Hardware-friendly?	**YES - Very**	**YES**	**CAUTION - Expensive**
In this codebase?	**YES**	**YES**	**NO**

04

Inference & Convergence

04 - Inference (Retrieval) - How It Works

```
s_i  =  sign(  Sum_j  W_ij * s_j  )    =    sign( h_i )
```

Apply to one neuron at a time (async) or all at once (sync) - repeat until fixed point

Asynchronous Update	Synchronous Update
One random neuron updated per step	ALL neurons updated simultaneously
Energy E = -(1/2) s^T W s monotonically decreases	Maps to single clock-cycle RTL update
Convergence to fixed point GUARANTEED	Can oscillate (2-cycles) - not always a fixed pt
Used in Python + C simulation	Used in hardware FSM (COMPUTE state)
steps - 10-20 - N for reliable recall	Fixed-point check after each full round

```
RTL FSM (update_ctrl.sv):   IDLE  -  COMPUTE  -  CHECK_FP  -  DONE
CHECK_FP: compare state_before vs state_after - equality signals convergence
```

04 - Lyapunov Energy Function - Why Convergence Is Guaranteed

$$E(s) = -(1/2)\ s^T W s = -(1/2)\ \sum_{i,j}\ W_{ij} * s_i * s_j$$

Bounded Below

W is finite & symmetric - E has a global minimum

Non-Increasing

Each async update satisfies: Delta_E = -(s_i_new - s_i)*h_i <= 0

Fixed Points = Minima

A state with no lower-energy neighbor is a fixed point

Finite State Space

{-1,+1}^N has 2^N states - must reach minimum in finite steps

- Spurious Attractors

Stored patterns are designed to be energy minima, but other minima exist too:
- Inverted patterns (-) are always fixed points
- Mixture states (average of k stored patterns) emerge near capacity

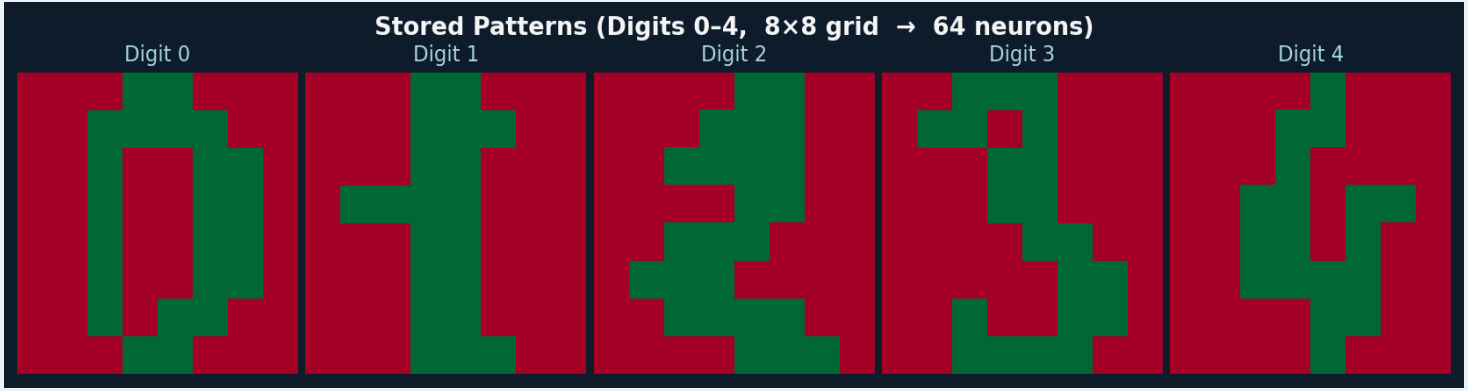
The network can 'recall' these instead of a stored pattern - happens more near - - 0.138

05

MNIST Visualization - Store, Corrupt & Retrieve

05 - Stored Digit Patterns (sklearn 8-8 Digits, binarised to -1)

5 digit templates (digits 0-4) stored as bipolar vectors of length 64. Green = +1 (active neuron), Red = -1 (inactive neuron).



N = 64 neurons

8-8 pixel grid per digit

P = 5 patterns

Digits 0, 1, 2, 3, 4

Hebbian training

W = (1/64) * Sum_mu xi_mu (xi_mu)^T

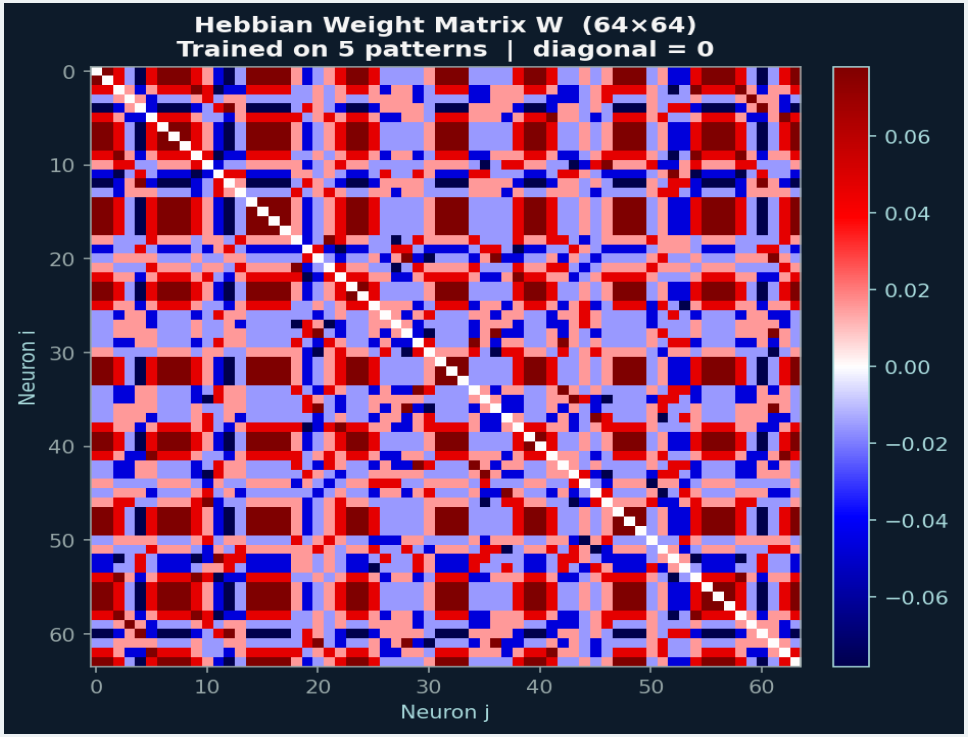
Capacity

~9 patterns before recall degrades

Training: Hebbian rule applied once to all 5 patterns simultaneously - W is 64-64 symmetric matrix with W_ii = 0

05 - Learned Weight Matrix W (64-64 Hebbian)

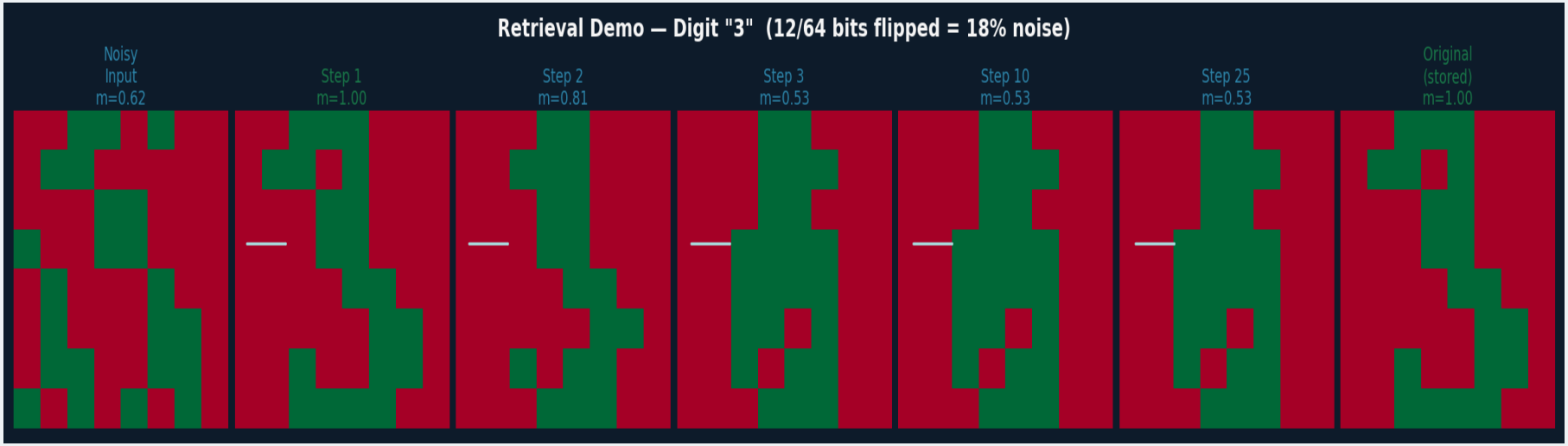
Red = strong negative weight (neurons tend to disagree). Blue = strong positive weight (neurons tend to agree). Diagonal is forced to zero - no self-connections.




```
W_ij = (1/N) * sum_mu  xi_i^mu * xi_j^mu    |    W_ij = W_ji    |    W_ii = 0
```

05 - Retrieval Demo - Digit "3" with 12/64 bits corrupted (~19% noise, 25 sync steps)

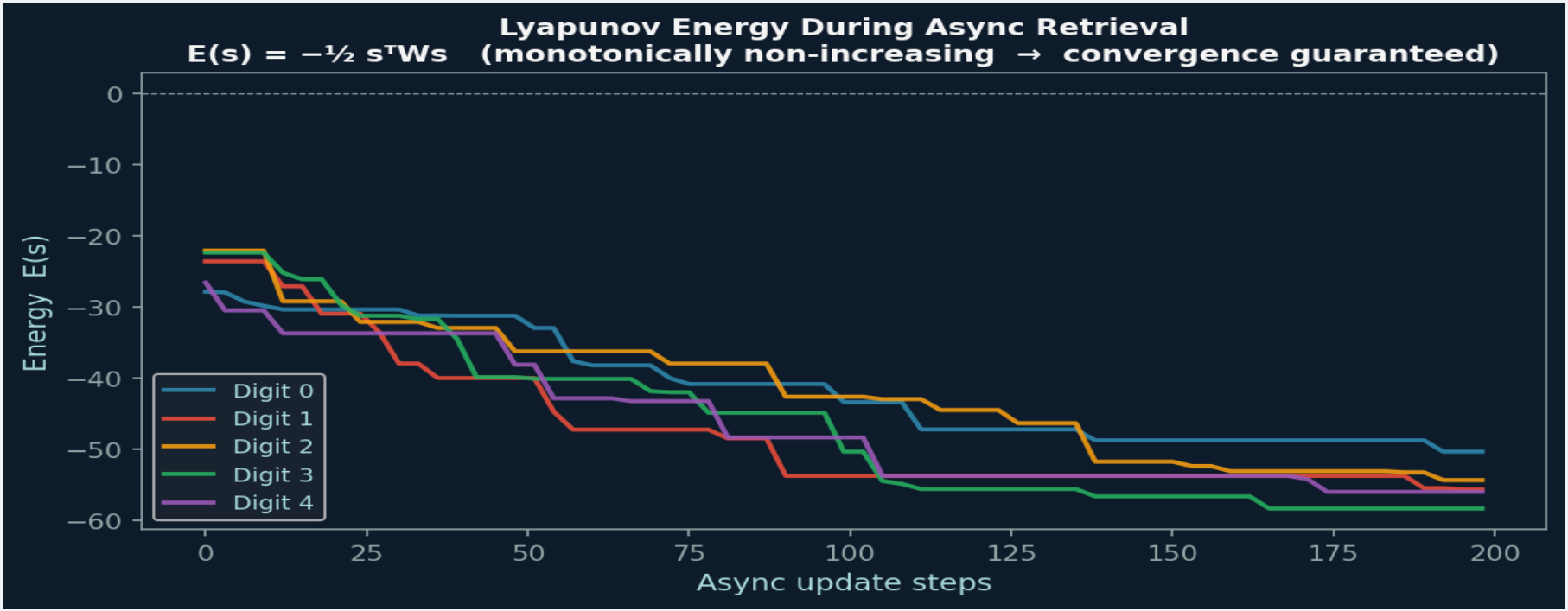
Starting from a noisy probe (19% of bits flipped), synchronous updates converge to the stored "3" pattern within a few steps. Overlap m = (1/N) xi.s -> 1.0.



Noisy Input	Steps 1-3	Step 10-25	Original
19% bits flipped from stored digit	Most neurons snap to correct state fast	Residual errors corrected, m - 1.0	Ground truth stored pattern

05 - Lyapunov Energy During Async Retrieval (5 digits, 8 bits corrupted each)

Energy decreases monotonically with each asynchronous update step - provably. All five digit patterns converge to stable energy minima within ~100 steps for N=64.



`E(s) = -(1/2) s^T W s` | `Delta_E = -(s_i_new - s_i)*h_i <= 0` | `Finite state space -> guaranteed convergence`

Summary & Next Steps

Code	5 Python modules + 3 C files covering full train-RTL pipeline. Well-parameterized by N.
Training	Hebbian (simple, one-shot) and Storkey (higher cap, sequential) both implemented. Hebbian preferred for HW.
Inference	Async update: convergence guaranteed via Lyapunov energy. Sync update: used in RTL, may 2-cycle.
Capacity	Hard limit ~0.138N patterns. At N=64 - ~9 patterns. Truth-table synthesis only feasible for N-20.
MNIST Demo	5 digits stored in 64-neuron net. 19% corrupted input reliably retrieved within 25 sync steps.